

AUTONOMOUS FACTOR RESEARCH

Codex Implementation Guide

What to borrow from “Beyond Prompting: An Autonomous Framework for Systematic Factor Investing via Agentic AI”

Purpose

This document converts the paper into a concrete engineering blueprint for the Anthelion take-home. It separates (1) ideas directly supported by the paper from (2) recommended adaptations for the 2010-2016 NYSE + fundamentals + news assignment. The goal is not to reproduce the paper exactly; the goal is to build a smaller, auditable, reproducible autonomous research loop that Codex can implement.

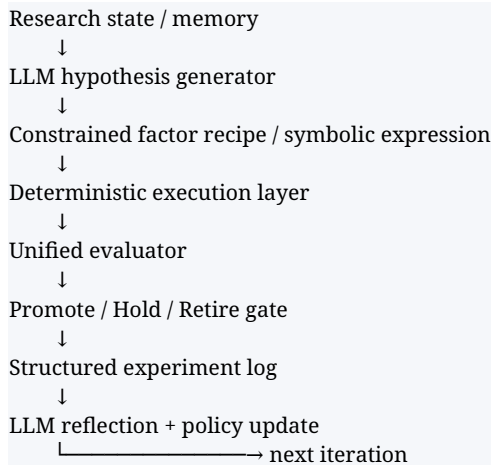
Primary source: Huang & Fan (2026), arXiv:2603.14288v1

Contents

1. What the paper actually does
2. The design principles worth copying
3. Recommended adaptation for the Anthelion project
4. Core schemas for Codex to implement
5. Evaluation and promotion gates
6. Memory, mutation, and iteration logic
7. Repository architecture and run sequence
8. What not to copy blindly
9. Definition of done

1. What the paper actually does

The paper reframes factor discovery as a closed-loop research process rather than a one-shot prompting task. In every iteration, the system generates factor hypotheses, constructs the signals, evaluates them under one common protocol, applies a gate, records the result, and uses the accumulated evidence to change what it searches next. The paper formalizes this as $H_k \rightarrow F_k \rightarrow M_k \rightarrow D_k$. (Paper Sec. 3.1, pp. 8-9.)



The paper’s architecture has five operational modules: (1) constrained hypothesis generation, (2) deterministic execution, (3) unified evaluation, (4) transparent gatekeeping, and (5) memory / policy update. Candidate factors are symbolic compositions of a fixed variable set and fixed operator set, with bounded expression complexity. (Paper Sec. 3.2, pp. 9-10.)

The key distinction is that the LLM proposes research logic, while code performs the numerical work. Cross-sectional transforms are applied date-by-date; time-series transforms are applied within each asset history. This is a strong reproducibility pattern and should be copied directly.

Main lesson

Do not build “an LLM that writes arbitrary pandas code.” Build “an LLM that chooses among a constrained factor language, while deterministic code owns data alignment, transforms, backtesting, and validation.”

2. The design principles worth copying

2.1 Constrained autonomy

The agent can explore broadly, but only inside explicit scientific constraints: fixed data universe, admissible operators, bounded expression depth, and no-lookahead rules. The paper uses this to keep factors interpretable and auditable. (Paper Sec. 3.1-3.2, pp. 8-10.)

Allowed primitives: return, price, volume, turnover, volatility, market state, ...
 Allowed operators: lag, rolling_mean, rolling_std, rank, zscore, winsorize, arithmetic combinations, simple interactions
 Forbidden: forward returns, future windows, global full-sample normalization, arbitrary external data, unrestricted Python execution
 Complexity: explicit max depth / max operator count

2.2 Economic rationale before empirical reward

The appendix frames the agent as Reasoning + Action: an economic rationale is formed together with the factor expression before the backtest result is observed. The paper treats this as an “economic regularizer” against purely combinatorial data mining. (Appendix A, pp. 44-45; Sec. 6.1, pp. 28-29.)

For this project, require every candidate to contain a one- or two-sentence mechanism: what behavioral, risk, information-diffusion, liquidity, valuation, or regime effect could make the signal predict future returns? This rationale is metadata, not proof; the factor still has to pass empirical gates.

2.3 One common evaluator for every candidate

The paper explicitly rejects candidate-specific scoring. Every factor passes through the same evaluator and the same promotion logic. This is critical for avoiding moving-goalpost research. (Paper Sec. 3.2-3.3, pp. 9-13.)

2.4 Frozen out-of-sample policy

The paper freezes the agent state and factor library before the OOS period. During OOS, results are reported but cannot be used to alter thresholds, re-rank factors, or update the search policy. (Paper Sec. 3.3 and Sec. 6.4, pp. 11-13 and 30.)

Direct project implication

For the Anthelion dataset, 2016 should be physically excluded from the research loop. The loop may adapt within 2010-2015, but 2016 is evaluated exactly once after the library and selection rules are frozen.

3. Recommended adaptation for the Anthelion project

The paper is a useful architecture reference, but the take-home has different data (prices + fundamentals + general news) and a much shorter sample (2010-2016). The implementation should therefore be smaller, stricter, and more transparent.

Component	Paper	Recommended Anthelion adaptation
Research data	CRSP price/volume style primitives	NYSE prices + point-in-time fundamentals + news-derived market/regime features
Agent output	Symbolic factor logic / generated factor action	Strict JSON FactorSpec; no arbitrary Python
Search	Memory-guided agentic search	Seed + constrained mutation + occasional exploration
Evaluation	Rank IC, t-stat, sorted portfolios, Sharpe	Same core metrics + walk-forward stability + turnover + redundancy
OOS	2021-2024 frozen OOS	2016 completely locked holdout
Combination	Linear + LightGBM	Start with equal-weight z-score or ridge; LightGBM optional only if time remains
Anti-mining	Economic rationale, t hurdle, multi-objective gate	Same + explicit correlation pruning + incremental IC / residual IC + complexity penalty

Recommended time split

Adaptive research environment (agent may learn from these results):

Fold 1: train / construct on 2010-2012 → validate on 2013

Fold 2: train / construct on 2010-2013 → validate on 2014

Fold 3: train / construct on 2010-2014 → validate on 2015

Final blind holdout:

2016 → read only after final factor library + thresholds are frozen

Important terminology: 2013-2015 are validation years inside the adaptive research environment, not the final holdout. Because the agent is allowed to learn from those results, they are part of the research process. Only 2016 is a true final blind test.

4. Core schemas for Codex to implement

The cleanest way to operationalize the paper is to make the agent communicate only through typed research objects. These objects create auditability, validation, and reproducibility.

4.1 FactorSpec

```
@dataclass
class FactorSpec:
    factor_id: str
    generation: int
    parent_ids: list[str]
    family: str          # price, fundamental, news, interaction
    hypothesis: str       # economic rationale

    base_feature: str     # e.g. return, earnings_yield, news_sentiment
    ts_operator: str | None # rolling_mean, delta, ewma, vol_adjust
    window: int | None
    cs_operator: str | None # rank, zscore, winsorize_rank
    interaction_feature: str | None
    direction: int        # +1 or -1

    complexity: int
    mutation_reason: str | None
```

The builder must reject any spec that references a non-whitelisted primitive/operator, exceeds maximum complexity, uses a future-looking transform, or cannot be computed point-in-time.

4.2 ExperimentRecord

```
@dataclass
class ExperimentRecord:
    factor_id: str
    generation: int
    parent_ids: list[str]
    hypothesis: str
    canonical_formula: str

    integrity_passed: bool
    integrity_issues: list[str]

    fold_metrics: dict
    mean_rank_ic: float
    ic_tstat: float
    positive_fold_count: int
    long_short_sharpe: float
    turnover: float
```

```

max_drawdown: float
redundancy_corr: float | None
residual_ic: float | None

decision: str          # promote / hold / retire
reasons: list[str]

```

4.3 ResearchState

```

@dataclass
class ResearchState:
    generation: int
    promoted_factor_ids: list[str]
    held_factor_ids: list[str]
    retired_factor_ids: list[str]
    family_summary: dict
    best_patterns: list[str]
    failed_patterns: list[str]
    unexplored_families: list[str]
    search_budget_remaining: int

```

Implementation rule

The LLM sees compact ResearchState + selected ExperimentRecords. It does not need the entire raw dataframe, and it does not calculate metrics itself.

5. Evaluation and promotion gates

The paper uses daily cross-sectional rank IC, its t-statistic, and a long-short sorted portfolio as core evidence. It also emphasizes multiple-testing concerns, economic viability, turnover, and OOS isolation. (Paper Sec. 3.3, pp. 11-13; Sec. 6.2-6.4, pp. 29-30.)

5.1 Integrity gate - before performance

- Signal at date t may depend only on information available at or before t .
- Cross-sectional rank/z-score/winsorization must be computed independently within each date.
- Rolling operations must be grouped by ticker and use historical observations only.
- Fundamentals must be delayed to a defensible availability date if the dataset does not provide an announcement timestamp.
- News features must use publication times/dates no later than the signal timestamp.
- Reject factors with pathological missingness, near-zero coverage, impossible values, or target leakage.

5.2 Common metrics

```

Statistical:
daily Spearman rank IC
mean IC
IC t-stat / ICIR
fraction of validation folds with correct sign

Economic:
quantile monotonicity
top-minus-bottom long-short return
annualized Sharpe
max drawdown
turnover
optional net Sharpe after a simple cost assumption

```

Novelty / redundancy:

cross-sectional signal correlation vs promoted library
 long-short return correlation vs promoted library
 residual / incremental IC after neutralizing against close existing signals

5.3 Gate logic

Do not reduce all evidence to one arbitrary weighted score. Use hard gates first, then tie-break among survivors. The paper's Promote / Hold / Retire design is directly useful, but this project should add explicit redundancy and stability rules.

```
def decide(record):
    if not record.integrity_passed:
        return RETIRE("integrity failure")

    if record.positive_fold_count < MIN_POSITIVE_FOLDS:
        return RETIRE("unstable across time")

    if record.mean_rank_ic < MIN_MEAN_IC:
        return RETIRE("weak predictive signal")

    if record.long_short_sharpe < MIN_SHARPE:
        return HOLD_OR_RETIRE("weak economic monetization")

    if record.turnover > MAX_TURNOVER:
        return HOLD_OR_RETIRE("implementation cost")

    if record.redundancy_corr > MAX_CORR and record.residual_ic < MIN_INCREMENTAL_IC:
        return RETIRE("redundant with existing factor")

    return PROMOTE
```

The paper adopts a stricter $|t| > 3$ discussion for multiple-testing concerns in Sec. 6.2. With only 2010-2015 research data, do not mechanically insist on exactly the same threshold if it leaves no usable candidates; pre-commit your project thresholds and justify them. The key is that thresholds must not be tuned after looking at 2016.

6. Memory, mutation, and iteration logic

The paper's most important "agentic" feature is not the LLM itself; it is the feedback loop. After every round, structured outcomes become the next round's state. This should be implemented explicitly rather than hidden inside chat history.

6.1 Exploration vs exploitation

Each new generation should contain BOTH:

EXPLOITATION (about 60-80%)

- change lookback window around successful parents
- add/remove smoothing
- volatility-adjust a successful price signal
- combine a successful base signal with news regime
- simplify an over-complex survivor

EXPLORATION (about 20-40%)

- new primitive family
- new economic hypothesis
- under-tested fundamental variable

- new news/regime transformation

6.2 Example trajectory

Generation 0:

F001 = rank(return_20d)

→ good IC, moderate Sharpe, weak in negative-news regime

Generation 1 (mutation):

F031 = rank(return_20d / vol_20d)

→ improved stability, lower turnover

Generation 2 (cross-domain interaction):

F067 = zscore(vol_adjusted_momentum) × negative_news_regime

→ test whether news conditions momentum strength

Generation 3:

compare smoothed vs unsmoothed interaction

keep only if incremental IC survives and correlation to parent is not too high

The paper's Table IX shows why explicit redundancy pruning matters: several final factors are near-duplicates of turnover or relative-turnover constructions. Treat that as a useful warning. Your library should prefer one robust representative of a signal family over many parameter variants.

6.3 Candidate-generation prompt contract

SYSTEM / CONTRACT FOR LLM RESEARCHER

You are a hypothesis generator, not a backtester.

You may only emit valid FactorSpec JSON from the supplied primitive and operator catalog.

Use the research history to avoid duplicates.

Every proposal must include:

- 1) economic hypothesis,
- 2) parent or exploration rationale,
- 3) exact FactorSpec,
- 4) why the proposal could improve on prior evidence.

Do not reference OOS/holdout results.

Do not invent columns or functions.

Prefer simple interpretable expressions.

7. Repository architecture and run sequence

```
src/
├── data/
│   ├── loader.py
│   ├── preprocess.py
│   └── point_in_time.py
├── factors/
│   ├── primitives.py
│   ├── transforms.py
│   ├── schema.py
│   └── builder.py
├── evaluation/
│   ├── integrity.py
│   ├── ic.py
│   ├── portfolio.py
│   ├── validation.py
│   └── redundancy.py
├── research/
│   └── generator.py
```

```

├── mutator.py
├── selector.py
├── memory.py
├── loop.py
├── utils/
│   └── logging.py
├── scripts/
│   ├── prepare_data.py
│   ├── run_loop.py
│   └── final_holdout.py
├── experiments/
│   ├── candidates.csv
│   ├── generations.csv
│   └── runs/
├── outputs/
│   ├── figures/
│   └── tables/

```

Run sequence

1. `prepare_data.py`: create `research_2010_2015.parquet` and `holdout_2016.parquet`; the research loop never imports the holdout file.
2. `run_loop.py`: generate seeds, build signals, run integrity checks, evaluate walk-forward folds, gate candidates, update memory, mutate/promote for several generations.
3. `freeze`: persist final config, factor library, thresholds, random seed, code commit hash, and experiment log.
4. `final_holdout.py`: load frozen library + 2016 only, compute final report exactly once.
5. `report`: show the research trajectory, including what was proposed, promoted, held, retired, and why.

Minimal loop pseudocode

```

state = initialize_state(seed_specs)

for generation in range(cfg.num_generations):
    candidates = generator.propose(state)
    records = []

    for spec in candidates:
        factor = builder.build(spec, research_panel)
        integrity = integrity_checker.check(spec, factor)

        if not integrity.passed:
            records.append(retired_record(spec, integrity))
            continue

        metrics = validator.evaluate_walk_forward(factor)
        novelty = redundancy_checker.evaluate(factor, state.promoted_library)
        record = selector.make_record(spec, metrics, novelty)
        record.decision, record.reasons = selector.decide(record)
        records.append(record)

    experiment_store.append(records)
    state = memory.update(state, records)

freeze(state, config, library)
# only now is final_holdout.py allowed to read 2016

```

8. What not to copy blindly

Do not copy	Why	Better choice for this take-home
Arbitrary LLM-generated Python	Hard to validate; easy to leak; brittle	LLM emits FactorSpec only; deterministic builder executes
Complex aggregation immediately	Adds another source of overfit	Start equal-weight / ridge; add LightGBM only as optional robustness extension
Many near-duplicate factors	Inflates library without adding information	Correlation pruning + residual IC / incremental value test
One full-sample Sharpe	Adaptive search will overfit it	Walk-forward research validation + untouched 2016
Changing thresholds after seeing results	Turns “evaluation” into hidden optimization	Pre-commit thresholds in config and log config version
Treating economic story as evidence	Narratives can be retrofitted	Use rationale only as search regularization; empirical gate decides
Overly complex symbolic formulas	Hard to audit; likely data mining	Complexity cap and “simpler wins if performance is similar”

9. Definition of done for Codex

Codex should consider the autonomous factor-research system complete only when all of the following are true:

- ☐ A deterministic research panel exists for 2010-2015, and a separate holdout file exists for 2016.
- ☐ A typed FactorSpec grammar restricts primitives, operators, windows, and complexity.
- ☐ The factor builder is deterministic and has unit tests for cross-sectional and time-series transforms.
- ☐ Integrity checks can detect at least one intentionally constructed look-ahead factor and reject it.
- ☐ Every factor is evaluated under the exact same walk-forward protocol.
- ☐ Each experiment writes a structured record with definition, hypothesis, metrics, decision, and reason.
- ☐ The search process runs for multiple generations and uses prior outcomes to alter future proposals.
- ☐ The library removes or rejects highly redundant factors.
- ☐ News enters the search space either as direct signal features or as market/regime conditioning variables.
- ☐ 2016 is not read by generator, evaluator, selector, or memory modules during research.
- ☐ final_holdout.py can reproduce one frozen 2016 evaluation from the saved library and config.
- ☐ The output contains a trajectory table and factor lineage showing proposed / survived / killed candidates and reasons.

Suggested project positioning

“A constrained, memory-guided autonomous factor research agent that proposes economically motivated factor recipes, executes them through a deterministic point-in-time engine, evaluates them under fixed temporal validation, and uses structured empirical feedback to guide subsequent

exploration and exploitation.”

Source notes

Paper-derived claims in this guide are based on Huang & Fan (2026), “Beyond Prompting: An Autonomous Framework for Systematic Factor Investing via Agentic AI,” arXiv:2603.14288v1. Key source sections used: Sec. 3.1-3.4 (closed loop, factor grammar, evaluator, gate, memory); Sec. 4.2-4.3 (predictor set and reproducibility); Sec. 6.1-6.5 (economic regularization, multiple testing, temporal isolation, factor anatomy); Sec. 7 (robustness); Appendix A-C (ReAct framing, aggregation, OOS safeguards). Recommendations labeled as Anthelion adaptations are implementation choices for this take-home, not claims that the paper itself used those exact choices.